LESSON PLAN · PYTHON (GR 6–8) BEGINNER: PYTHON GAME CODERS

Lesson 6: Random and Games

Module 2 | 60 minutes | random, game loops | Grades 6–10 · ages 11–15

Objective: Use the random module to roll dice and pick choices, and build a guessing game with a while loop, hints and an attempt counter.

Success criteria

- I can import the random module.
- I can use randint and choice in a program.
- I can build a game loop with a while loop and a counter.
- I can give hints with if / elif / else.

Key vocabulary

- **module**: A toolbox of extra commands, opened with import.
- **import**: Loads a module so you can use it.
- **randint(a, b)**: A random whole number from a to b, both included.
- **choice(list)**: Picks one random item from a list.
- **game loop**: The repeating part of a game: ask, check, respond.
- **counter**: A variable that counts, like attempts += 1.

You will need

- Projector with Kitabash (lesson video, Python Lab)
- Devices: 1 per learner, with a keyboard
- Printed worksheet (optional)
- Project files: dice_roller.py

Timed lesson plan (60 min)

Time	Stage	What happens
5 min	**Warm-up game: Higher or Lower**	Play Higher or Lower (unplugged) in the chat. One learner keeps count on the board. Discuss the best first guess.
5 min	**Hook and video**	Show the hook. Prediction poll: “What number will it print?” Run it live to see who guessed right. Play the video with captions.
10 min	**Live demo and build-along**	Run the live demo, then everyone builds the dice roller. Thumbs check: “Who rolled doubles?”
8 min	**Challenge, predict and match**	Dice roller challenge, the randint predict and the match. Quick competition: the class rolls together and the highest total wins.
15 min	**Project in breakout pairs**	Pairs build Guess the Number; each partner plays the other’s game and counts their tries.

6 min	**Debug race**	Race to fix “the crashing guesser”: two bugs. First to explain why else was wrong wins.
7 min	**Show-and-tell tournament**	Two volunteers share their game; the class plays together by voting on each guess. Shout-out for the fewest tries.
4 min	**Wrap-up and exit ticket**	Fist-to-five on the success criteria. Exit ticket: secret = random.randint(1, 100).

Unplugged warm-up

Higher or Lower (5 min game): the teacher secretly writes a number from 1 to 50. Learners take turns guessing in the chat; the teacher replies only “too high” or “too low”, and one learner keeps the attempts count on the board. Then ask: what is the cleverest first guess? (25, the middle.)

Teaching the idea

Modules are toolboxes of extra commands; import random gives us randint and choice. randint(a, b) includes both ends. A game loop repeats ask, check, hint until a win or a limit. Combining while with and lets us stop for two reasons: the guess is right OR the tries are used up. In the Python Lab the random numbers are real for the player, but when the Lab checks your work it uses fixed test answers, so make your game follow the rules exactly.

Analogy: random.randint is a dice you never have to pick up, and random.choice is like pulling a chit out of a bowl at a class party.

Live demo: 1) Type import random and print(random.randint(1, 6)). Run it five times and write the numbers on the board. 2) Delete the import line and run: show the NameError, then put it back. 3) Type snacks = ["samosa", "jalebi", "dhokla", "idli"] and print(random.choice(snacks)). Run it a few times. 4) Set secret = 23 (fixed, so everyone can follow) and type the if / elif / else hint code. Run with 30, 10 and 23. 5) Wrap it in while guess != secret: with guess = 0 above, and add attempts += 1. Play one round with the class calling out guesses. 6) Change secret back to random.randint(1, 50) and play again.

Questions to ask

Ask	Listen for
What does import random do?	It opens the random toolbox so we can use randint and choice.
Can random.randint(1, 6) give 6?	Yes, both ends are included.
Why do we need attempts += 1 inside the loop?	To count each guess, so we can stop at 7 and report the tries.
Why did the guess crash when we compared it to secret?	input() gives text; we need int() to compare numbers.
What is a good first guess from 1 to 50?	25, the middle, because it cuts the options in half.

Differentiation

Support
Let learners start with a fixed secret = 23 so they can test their hints, then swap in randint at the end.

Stretch
Add difficulty levels: easy (1–20, 7 tries), hard (1–100, 7 tries). Or give a “very close!” hint when the guess is within 3.

Common misconceptions

Misconception	How to fix it
randint(1, 6) never gives 6	Both ends are included, unlike range.
You can use random without importing it	You must write import random first.
The loop stops when attempts reaches 7 even without a condition	You must add attempts < 7 to the while condition.
else only runs when the guess is too low	else runs for everything not caught above, including the right answer.

Assessment

Use the Python Lab results (auto-checked), the quick checks and the Probe answers. Written answers arrive in your review queue.

Exit ticket
Write one line that picks a random number from 1 to 100 and stores it in a variable called secret.

Homework
Play Guess the Number with someone at home. Who needs the fewest tries? Then add a “very close!” hint.

Answer key — Kitabash activities

1. Python Lab: dice roller (perform · python lab)

Goals checked: Uses the random module · Prints the total · Checks for doubles · Runs with no errors

One solution:

```
# Dice roller
import random

die1 = random.randint(1, 6)
die2 = random.randint(1, 6)
print("Die 1:", die1)
print("Die 2:", die2)
print("Total:", die1 + die2)

if die1 == die2:
    print("Doubles! Roll again!")

snack = random.choice(["samosa", "jalebi", "dhokla", "idli"])
print(f"Today's lucky snack is {snack}!")
```

2. Predict the output: what can randint give? (perform · code output)

3. Match each line of code to what it does (perform · matching)

import random → Opens the random toolbox · random.randint(1, 6) → A whole number from 1 to 6 · random.choice(moves) → Picks one item from a list · attempts += 1 → Adds one to the try counter · while guess != secret: → Keeps going until the guess is right

4. Project: Guess the Number (perform · python lab)

Goals checked: Says “Too low!” for a low guess · Says “Too high!” for a high guess · Stops after 7 tries · Uses random, while and if

Example program:

```
# Guess the Number
# Made by: Meera
import random

secret = random.randint(1, 50)
attempts = 0
guess = 0

print("=== GUESS THE NUMBER ===")
print("I am thinking of a number from 1 to 50.")
print("You have 7 tries. Good luck!")

while guess != secret and attempts < 7:
    guess = int(input(f"Try {attempts + 1}: "))
    attempts += 1
    if guess < secret:
        print("Too low!")
    elif guess > secret:
        print("Too high!")
    else:
        print(f"You got it in {attempts} tries!")

if guess != secret:
    print(f"Out of tries! The number was {secret}.")
```

5. Debug it: the crashing guesser (probe · python lab)

Goals checked: Runs without crashing · Ends with the win message · Only says Too low when it is low

One solution:

```
import random
secret = random.randint(1, 10)
guess = 0
tries = 0
while guess != secret:
    guess = int(input("Guess 1-10: "))
    tries += 1
    if guess > secret:
        print("Too high!")
    elif guess < secret:
        print("Too low!")
print("You got it in", tries, "tries!")
```

Answer key — worksheet

1. **Key words:** module = A toolbox of extra commands, opened with import. · import = Loads a module so you can use it. · randint(a, b) = A random whole number from a to b, both included. · choice(list) = Picks one random item from a list.
2. **Plan your program:** Goals: Uses the random module; Prints the total; Checks for doubles; Runs with no errors. One solution: # Dice roller import random die1 = random.randint(1, 6) die2 = random.randint(1, 6) print("Die 1:", die1) print("Die 2:", die2) print("Total:", die1 + die2) if die1 == die2: print("Doubles! Roll again!") snack = random.choice(["samosa", "jalebi", "dhokla", "idli"]) print(f"Today's lucky snack is {snack}!")
3. **Predict the output: what can randint give?:** A. 40
4. **Match each line of code to what it does:** import random → Opens the random toolbox · random.randint(1, 6) → A whole number from 1 to 6 · random.choice(moves) → Picks one item from a list · attempts += 1 → Adds one to the try counter · while guess != secret: → Keeps going until the guess is right
5. **Plan your program:** Goals: Runs without crashing; Ends with the win message; Only says Too low when it is low. One solution: import random secret = random.randint(1, 10) guess = 0 tries = 0 while guess != secret: guess = int(input("Guess 1-10: ")) tries += 1 if guess > secret: print("Too high!") elif guess < secret: print("Too low!") print("You got it in", tries, "tries!")

Kitabash original curriculum.